TB Burden Reporting Analytics

Seif H. Kungulio

TB Burden Reporting Analytics

Seif H. Kungulio

March 6, 2026

Contents

Introduction **1**

Database Creation **2**

Environment Setup . . . 2
Database Connection . . . 2
Source File Overview . . . 3
Schema Strategy . . . 4
Prerequisites . . . 4
Reset Existing Objects . . . 5
Load the CSV . . . 5
Cleaned Staging Table . . . 7
Data Quality Checks . . . 9
Row Count Validation . . . 9
Duplicate Detection . . . 9
Missing Value Validation . . . 10
Dimension Tables . . . 10
Country dimension . . . 10
Date dimension . . . 11
Methodology dimension . . . 11
Fact Table . . . 12
Load Dimensions . . . 13
Country dimension . . . 13
Date dimension . . . 14
Methodology dimension . . . 14
Load Fact Table . . . 15
Performance Indexes . . . 17
Reporting Views . . . 17
Country-Year Reporting View . . . 17
Regional Summary View . . . 19
Validation Queries . . . 19
Row Count Reconciliation . . . 19
Fact Table Integrity Check . . . 21
Sample Reporting Queries . . . 21
Country Trend Analysis . . . 21
Top Countries Analysis . . . 22

Regional Summary Analysis 23
Regional Averages Analysis 23

Conclusion **25**

Introduction

Modern public health analysis depends heavily on the ability to transform complex datasets into structured, reliable reporting systems that support data-driven decision-making. This project demonstrates how global tuberculosis (TB) burden data can be transformed into a scalable analytical environment by designing a structured reporting database using PostgreSQL and a dimensional modeling approach. The source dataset contains country-level annual estimates for TB prevalence, incidence, mortality, HIV-associated TB burden, and case detection rates across multiple regions and years. These metrics provide valuable insight into how TB affects populations globally and how disease patterns evolve over time.

To support efficient reporting and analytical exploration, the project implements a star schema data model built around a clearly defined analytical grain of one record per country per year. This structure organizes descriptive geographic information, time attributes, and estimation methodologies into dedicated dimension tables, while quantitative disease indicators are stored within a centralized fact table. By separating descriptive attributes from measurable indicators, the design reduces redundancy, simplifies query logic, and enables efficient aggregation of health metrics across countries, regions, and time periods.

The workflow also demonstrates several professional data engineering practices commonly used in reporting and analytics environments. These include building a reproducible pipeline in RMarkdown, establishing secure database connectivity, creating a raw ingestion layer to preserve source data, implementing a cleaned staging layer for transformation and validation, and loading the final data into a dimensional warehouse structure. Data quality checks are incorporated throughout the process to verify row consistency, confirm the integrity of the reporting grain, and ensure that key attributes such as country identifiers and reporting years are complete before the analytical model is built.

The resulting reporting model supports a wide range of analytical use cases. Analysts can explore country-level TB trends over time, compare disease burden across regions, evaluate case detection performance, and identify countries with the highest incidence rates. Because the dataset is structured as a star schema, it can be easily connected to business intelligence platforms or dashboard tools to enable interactive reporting and public health monitoring. Overall, the project demonstrates how raw epidemiological data can be transformed into a maintainable and performance-optimized analytical system designed for reporting, trend analysis, and strategic decision support.

Database Creation

Environment Setup

The setup chunk prepares the RMarkdown execution environment so the entire project can run in a reproducible and controlled manner. In this step, the script configures chunk options, ensures that the required packages are installed, and loads the libraries necessary for database connectivity, data manipulation, and file ingestion. Establishing these dependencies early prevents runtime errors and ensures that the workflow behaves consistently regardless of where it is executed. This preparation step reflects sound analytical engineering practices because it removes manual setup requirements and makes the project portable for collaborators or reviewers.

```
knitr::opts_chunk$set(echo = TRUE, message = FALSE, warning = FALSE)

options(dplyr.print_min = Inf, dplyr.print_max = Inf)
options(repos = c(CRAN = "https://cloud.r-project.org"))

required_pkgs <- c("DBI", "RPostgres", "ini", "readr", "dplyr", "glue")

for (pkg in required_pkgs) {
  if (!requireNamespace(pkg, quietly = TRUE)) {
    install.packages(pkg)
  }
  library(pkg, character.only = TRUE)
}
```

The workflow begins by establishing a reproducible R environment configured with the required libraries for database connectivity, configuration management, data ingestion, and transformation. Preparing the execution environment at the outset ensures that the entire analytics pipeline can run consistently across systems without dependency conflicts. This approach reflects strong reproducibility practices and demonstrates familiarity with professional analytics workflows where projects must be portable and maintainable.

Database Connection

The database connection chunk establishes communication between R and the PostgreSQL database used for the reporting project. Credentials are read from an external configuration file rather than being embedded directly in the code, which represents a safer and more professional approach to handling sensitive connection details. Once the connection is created, it becomes the active session through which all SQL commands are executed throughout the workflow. This step demonstrates the integration of analytical tools with relational database systems, a common requirement in modern reporting and analytics environments.

```
config <- read.ini("resources/db_config.ini")
db <- config$postgresql

tryCatch({
  con <- dbConnect(
    RPostgres::Postgres(),
    host = db$host,
    dbname = db$database,
    user = db$user,
    password = db$password,
    port = as.integer(db$port)
```

```
  )
}, error = function(e) {
  stop("Database connection failed: ", e$message)
})

knitr::opts_chunk$set(connection = con)
```

A secure connection to the PostgreSQL database is established using parameters stored in an external configuration file rather than hard-coded credentials. By separating connection details from the codebase, the project follows secure development practices and supports deployment across different environments. Once connected, R is able to send SQL commands directly to the database, enabling seamless integration between the analytical scripting environment and the relational database system.

Source File Overview

The source inspection chunk performs an initial exploration of the incoming TB burden dataset before it is ingested into the database. By previewing several rows of the file and listing the column structure, the code verifies that the dataset matches expectations and that the column headers are being interpreted correctly. This inspection step helps identify structural issues such as unexpected fields, formatting problems, or inconsistencies in the dataset. Conducting this check before ingestion reflects responsible data validation practices and helps ensure that downstream transformations are built on a reliable data foundation.

```
tb_raw_preview <- readr::read_csv("resources/TB_Burden_Country.csv", show_col_types = FALSE, n_max = 5)
colnames(tb_raw_preview)
```

```
##  [1] "Country or territory name"
##  [2] "ISO 2-character country/territory code"
##  [3] "ISO 3-character country/territory code"
##  [4] "ISO numeric country/territory code"
##  [5] "Region"
##  [6] "Year"
##  [7] "Estimated total population number"
##  [8] "Estimated prevalence of TB (all forms) per 100 000 population"
##  [9] "Estimated prevalence of TB (all forms) per 100 000 population, low bound"
## [10] "Estimated prevalence of TB (all forms) per 100 000 population, high bound"
## [11] "Estimated prevalence of TB (all forms)"
## [12] "Estimated prevalence of TB (all forms), low bound"
## [13] "Estimated prevalence of TB (all forms), high bound"
## [14] "Method to derive prevalence estimates"
## [15] "Estimated mortality of TB cases (all forms, excluding HIV) per 100 000 population"
## [16] "Estimated mortality of TB cases (all forms, excluding HIV), per 100 000 population, low bound"
## [17] "Estimated mortality of TB cases (all forms, excluding HIV), per 100 000 population, high bound
## [18] "Estimated number of deaths from TB (all forms, excluding HIV)"
## [19] "Estimated number of deaths from TB (all forms, excluding HIV), low bound"
## [20] "Estimated number of deaths from TB (all forms, excluding HIV), high bound"
## [21] "Estimated mortality of TB cases who are HIV-positive, per 100 000 population"
## [22] "Estimated mortality of TB cases who are HIV-positive, per 100 000 population, low bound"
## [23] "Estimated mortality of TB cases who are HIV-positive, per 100 000 population, high bound"
## [24] "Estimated number of deaths from TB in people who are HIV-positive"
## [25] "Estimated number of deaths from TB in people who are HIV-positive, low bound"
## [26] "Estimated number of deaths from TB in people who are HIV-positive, high bound"
## [27] "Method to derive mortality estimates"
```

```
## [28] "Estimated incidence (all forms) per 100 000 population"
## [29] "Estimated incidence (all forms) per 100 000 population, low bound"
## [30] "Estimated incidence (all forms) per 100 000 population, high bound"
## [31] "Estimated number of incident cases (all forms)"
## [32] "Estimated number of incident cases (all forms), low bound"
## [33] "Estimated number of incident cases (all forms), high bound"
## [34] "Method to derive incidence estimates"
## [35] "Estimated HIV in incident TB (percent)"
## [36] "Estimated HIV in incident TB (percent), low bound"
## [37] "Estimated HIV in incident TB (percent), high bound"
## [38] "Estimated incidence of TB cases who are HIV-positive per 100 000 population"
## [39] "Estimated incidence of TB cases who are HIV-positive per 100 000 population, low bound"
## [40] "Estimated incidence of TB cases who are HIV-positive per 100 000 population, high bound"
## [41] "Estimated incidence of TB cases who are HIV-positive"
## [42] "Estimated incidence of TB cases who are HIV-positive, low bound"
## [43] "Estimated incidence of TB cases who are HIV-positive, high bound"
## [44] "Method to derive TBHIV estimates"
## [45] "Case detection rate (all forms), percent"
## [46] "Case detection rate (all forms), percent, low bound"
## [47] "Case detection rate (all forms), percent, high bound"
```

```r
dim(tb_raw_preview)
```

```
## [1]  5 47
```

The TB burden dataset is initially inspected to confirm that the file structure aligns with expectations before ingestion begins. A preview of the dataset reveals a table containing forty-seven columns and several rows of data, allowing the analyst to verify column names, formatting, and general structure. Conducting this early validation reduces the likelihood of downstream failures and demonstrates a careful, quality-oriented approach to data ingestion.

Schema Strategy

The reporting model follows a star schema with the following components:

- `dim_country`: descriptive country and regional attributes
- `dim_date`: year-based reporting calendar
- `dim_methodology`: textual estimate derivation methods
- `fact_tb_burden`: all numeric country-year reporting measures

A dimensional modeling strategy is defined in order to organize the dataset into a reporting-friendly structure. The design introduces a star schema composed of three descriptive dimensions—country, date, and methodology—alongside a central fact table that stores the measurable TB burden metrics. Structuring the data in this manner supports efficient reporting, simplifies joins for analysts, and prepares the dataset for use in dashboards and business intelligence tools.

Prerequisites

The schema creation chunk creates a dedicated PostgreSQL schema designed specifically for the TB reporting data model. Organizing the project within its own schema improves clarity, prevents naming conflicts with other database objects, and allows all tables, views, and supporting structures related to the reporting

pipeline to be grouped together logically. Structuring a database environment in this way mirrors practices commonly used in enterprise data warehouses, where analytical assets are separated from transactional data to maintain governance and maintainability.

```
CREATE SCHEMA IF NOT EXISTS tb_reporting;
```

A dedicated schema named `tb_reporting` is created within PostgreSQL to store all tables and views associated with the project. Isolating reporting objects within their own schema improves maintainability, prevents naming conflicts, and mirrors common enterprise data warehouse practices. The environment therefore becomes organized and easier to manage as additional reporting assets are added.

Reset Existing Objects

The object reset chunk removes any previously created tables and views associated with the reporting model. This step ensures that the workflow can be executed repeatedly without conflicts caused by existing database objects. By clearing old objects before rebuilding them, the pipeline becomes fully reproducible and suitable for iterative development or testing. This design demonstrates an understanding of maintainable data engineering workflows, where pipelines must often be rerun as data evolves or improvements are introduced.

```
DROP VIEW IF EXISTS tb_reporting.vw_tb_country_year_report;
```

```
DROP VIEW IF EXISTS tb_reporting.vw_tb_region_year_summary;
```

```
DROP TABLE IF EXISTS tb_reporting.fact_tb_burden CASCADE;
```

```
DROP TABLE IF EXISTS tb_reporting.dim_methodology CASCADE;
```

```
DROP TABLE IF EXISTS tb_reporting.dim_date CASCADE;
```

```
DROP TABLE IF EXISTS tb_reporting.dim_country CASCADE;
```

```
DROP TABLE IF EXISTS tb_reporting.stg_tb_burden_country_clean CASCADE;
```

```
DROP TABLE IF EXISTS tb_reporting.raw_tb_burden_country CASCADE;
```

Existing tables and views related to the reporting model are removed prior to rebuilding the database structures. Resetting these objects ensures that the pipeline can be executed repeatedly without conflicts from previous runs. Maintaining a clean rebuild process demonstrates a production-style mindset where analytical pipelines must be reliable and rerunnable during iterative development.

Load the CSV

The raw data loading chunk ingests the TB burden CSV dataset into PostgreSQL as a raw landing table. During this process, the column names from the source file are converted into standardized SQL-friendly identifiers to simplify downstream transformations and queries. Creating a raw ingestion layer allows the original data to be preserved while still enabling analytical transformations to occur in later stages of the pipeline. This design mirrors modern data engineering architectures that separate raw ingestion from cleaned and modeled data layers.

```r
tb_raw <- readr::read_csv("resources/TB_Burden_Country.csv", show_col_types = FALSE)

names(tb_raw) <- c(
  "country_name_raw",
  "iso2_code_raw",
  "iso3_code_raw",
  "iso_numeric_code_raw",
  "region_raw",
  "year_raw",
  "estimated_total_population_raw",
  "prevalence_per_100k_raw",
  "prevalence_per_100k_low_raw",
  "prevalence_per_100k_high_raw",
  "prevalence_total_raw",
  "prevalence_total_low_raw",
  "prevalence_total_high_raw",
  "prevalence_method_raw",
  "mortality_excl_hiv_per_100k_raw",
  "mortality_excl_hiv_per_100k_low_raw",
  "mortality_excl_hiv_per_100k_high_raw",
  "deaths_excl_hiv_raw",
  "deaths_excl_hiv_low_raw",
  "deaths_excl_hiv_high_raw",
  "mortality_hiv_pos_per_100k_raw",
  "mortality_hiv_pos_per_100k_low_raw",
  "mortality_hiv_pos_per_100k_high_raw",
  "deaths_hiv_pos_raw",
  "deaths_hiv_pos_low_raw",
  "deaths_hiv_pos_high_raw",
  "mortality_method_raw",
  "incidence_per_100k_raw",
  "incidence_per_100k_low_raw",
  "incidence_per_100k_high_raw",
  "incident_cases_raw",
  "incident_cases_low_raw",
  "incident_cases_high_raw",
  "incidence_method_raw",
  "hiv_in_incident_tb_percent_raw",
  "hiv_in_incident_tb_percent_low_raw",
  "hiv_in_incident_tb_percent_high_raw",
  "incidence_hiv_pos_per_100k_raw",
  "incidence_hiv_pos_per_100k_low_raw",
  "incidence_hiv_pos_per_100k_high_raw",
  "incidence_hiv_pos_raw",
  "incidence_hiv_pos_low_raw",
  "incidence_hiv_pos_high_raw",
  "tbhiv_method_raw",
  "case_detection_rate_percent_raw",
  "case_detection_rate_percent_low_raw",
  "case_detection_rate_percent_high_raw"
)

DBI::dbWriteTable(
```

```
  con,
  name = DBI::Id(schema = "tb_reporting", table = "raw_tb_burden_country"),
  value = tb_raw,
  overwrite = TRUE,
  row.names = FALSE
)
```

The TB burden CSV file is loaded into PostgreSQL as a raw landing table. All fields are initially stored as text to preserve the source data exactly as received. Maintaining a raw layer protects the integrity of the original dataset and provides a stable reference point for auditing or troubleshooting later stages of the transformation process.

Cleaned Staging Table

The staging table transformation chunk converts the raw dataset into a structured and analysis-ready format. In this step, whitespace is removed, blank strings are converted to null values, and textual fields are cast into their correct data types such as integers or numeric measures. These transformations ensure that quantitative indicators such as TB prevalence, incidence, and mortality can be reliably aggregated and analyzed. By standardizing data types and cleaning inconsistent values, this step establishes a trusted staging layer that supports the dimensional model built later in the pipeline.

```
CREATE TABLE tb_reporting.stg_tb_burden_country_clean AS
SELECT
    NULLIF(TRIM(country_name_raw), '') AS country_name,
    NULLIF(TRIM(iso2_code_raw), '') AS iso2_code,
    NULLIF(TRIM(iso3_code_raw), '') AS iso3_code,
    NULLIF(TRIM(iso_numeric_code_raw::TEXT), '')::INT
    AS iso_numeric_code,
    NULLIF(TRIM(region_raw), '') AS region,
    NULLIF(TRIM(year_raw::TEXT), '')::INT AS year,
    NULLIF(TRIM(estimated_total_population_raw::TEXT), '')::BIGINT
    AS estimated_total_population,

    NULLIF(TRIM(prevalence_per_100k_raw::TEXT), '')::NUMERIC(18,2)
    AS prevalence_per_100k,
    NULLIF(TRIM(prevalence_per_100k_low_raw::TEXT), '')::NUMERIC(18,2)
    AS prevalence_per_100k_low,
    NULLIF(TRIM(prevalence_per_100k_high_raw::TEXT), '')::NUMERIC(18,2)
    AS prevalence_per_100k_high,
    NULLIF(TRIM(prevalence_total_raw::TEXT), '')::NUMERIC(18,2)
    AS prevalence_total,
    NULLIF(TRIM(prevalence_total_low_raw::TEXT), '')::NUMERIC(18,2)
    AS prevalence_total_low,
    NULLIF(TRIM(prevalence_total_high_raw::TEXT), '')::NUMERIC(18,2)
    AS prevalence_total_high,
    NULLIF(TRIM(prevalence_method_raw), '') AS prevalence_method,

    NULLIF(TRIM(mortality_excl_hiv_per_100k_raw::TEXT), '')::NUMERIC(18,2)
    AS mortality_excl_hiv_per_100k,
    NULLIF(TRIM(mortality_excl_hiv_per_100k_low_raw::TEXT), '')::NUMERIC(18,2)
    AS mortality_excl_hiv_per_100k_low,
    NULLIF(TRIM(mortality_excl_hiv_per_100k_high_raw::TEXT), '')::NUMERIC(18,2)
    AS mortality_excl_hiv_per_100k_high,
```

```
    NULLIF(TRIM(deaths_excl_hiv_raw::TEXT), ''):::NUMERIC(18,2)
    AS deaths_excl_hiv,
    NULLIF(TRIM(deaths_excl_hiv_low_raw::TEXT), ''):::NUMERIC(18,2)
    AS deaths_excl_hiv_low,
    NULLIF(TRIM(deaths_excl_hiv_high_raw::TEXT), ''):::NUMERIC(18,2)
    AS deaths_excl_hiv_high,

    NULLIF(TRIM(mortality_hiv_pos_per_100k_raw::TEXT), ''):::NUMERIC(18,2)
    AS mortality_hiv_pos_per_100k,
    NULLIF(TRIM(mortality_hiv_pos_per_100k_low_raw::TEXT), ''):::NUMERIC(18,2)
    AS mortality_hiv_pos_per_100k_low,
    NULLIF(TRIM(mortality_hiv_pos_per_100k_high_raw::TEXT), ''):::NUMERIC(18,2)
    AS mortality_hiv_pos_per_100k_high,
    NULLIF(TRIM(deaths_hiv_pos_raw::TEXT), ''):::NUMERIC(18,2)
    AS deaths_hiv_pos,
    NULLIF(TRIM(deaths_hiv_pos_low_raw::TEXT), ''):::NUMERIC(18,2)
    AS deaths_hiv_pos_low,
    NULLIF(TRIM(deaths_hiv_pos_high_raw::TEXT), ''):::NUMERIC(18,2)
    AS deaths_hiv_pos_high,

    NULLIF(TRIM(mortality_method_raw), '') AS mortality_method,

    NULLIF(TRIM(incidence_per_100k_raw::TEXT), ''):::NUMERIC(18,2)
    AS incidence_per_100k,
    NULLIF(TRIM(incidence_per_100k_low_raw::TEXT), ''):::NUMERIC(18,2)
    AS incidence_per_100k_low,
    NULLIF(TRIM(incidence_per_100k_high_raw::TEXT), ''):::NUMERIC(18,2)
    AS incidence_per_100k_high,
    NULLIF(TRIM(incident_cases_raw::TEXT), ''):::NUMERIC(18,2)
    AS incident_cases,
    NULLIF(TRIM(incident_cases_low_raw::TEXT), ''):::NUMERIC(18,2)
    AS incident_cases_low,
    NULLIF(TRIM(incident_cases_high_raw::TEXT), ''):::NUMERIC(18,2)
    AS incident_cases_high,

    NULLIF(TRIM(incidence_method_raw), '') AS incidence_method,

    NULLIF(TRIM(hiv_in_incident_tb_percent_raw::TEXT), ''):::NUMERIC(18,2)
    AS hiv_in_incident_tb_percent,
    NULLIF(TRIM(hiv_in_incident_tb_percent_low_raw::TEXT), ''):::NUMERIC(18,2)
    AS hiv_in_incident_tb_percent_low,
    NULLIF(TRIM(hiv_in_incident_tb_percent_high_raw::TEXT), ''):::NUMERIC(18,2)
    AS hiv_in_incident_tb_percent_high,

    NULLIF(TRIM(incidence_hiv_pos_per_100k_raw::TEXT), ''):::NUMERIC(18,2)
    AS incidence_hiv_pos_per_100k,
    NULLIF(TRIM(incidence_hiv_pos_per_100k_low_raw::TEXT), ''):::NUMERIC(18,2)
    AS incidence_hiv_pos_per_100k_low,
    NULLIF(TRIM(incidence_hiv_pos_per_100k_high_raw::TEXT), ''):::NUMERIC(18,2)
    AS incidence_hiv_pos_per_100k_high,
    NULLIF(TRIM(incidence_hiv_pos_raw::TEXT), ''):::NUMERIC(18,2)
    AS incidence_hiv_pos,
    NULLIF(TRIM(incidence_hiv_pos_low_raw::TEXT), ''):::NUMERIC(18,2)
```

```
    AS incidence_hiv_pos_low,
    NULLIF(TRIM(incidence_hiv_pos_high_raw::TEXT), ''))::NUMERIC(18,2)
    AS incidence_hiv_pos_high,

    NULLIF(TRIM(tbhiv_method_raw::TEXT), '') AS tbhiv_method,

    NULLIF(TRIM(case_detection_rate_percent_raw::TEXT), ''))::NUMERIC(18,2)
    AS case_detection_rate_percent,
    NULLIF(TRIM(case_detection_rate_percent_low_raw::TEXT), ''))::NUMERIC(18,2)
    AS case_detection_rate_percent_low,
    NULLIF(TRIM(case_detection_rate_percent_high_raw::TEXT), ''))::NUMERIC(18,2)
    AS case_detection_rate_percent_high
FROM tb_reporting.raw_tb_burden_country;
```

A cleaned staging table is generated from the raw dataset to standardize and prepare the data for analytical use. During this process, whitespace is trimmed, empty strings are converted to null values, and appropriate numeric data types are assigned to measurement fields such as prevalence, incidence, mortality, and case detection rates. Creating a staging layer ensures that the downstream data model relies on consistent and validated data rather than raw text inputs.

Data Quality Checks

Row Count Validation

The row count validation chunk checks whether the number of records in the cleaned staging table matches the number of records loaded into the raw ingestion table. This reconciliation step confirms that no rows were unintentionally lost during the transformation process. Maintaining identical row counts between ingestion and staging layers is an important quality assurance practice because it verifies that the cleaning logic preserved the full dataset. This type of validation is commonly used in production reporting pipelines to ensure that transformations do not introduce data loss.

```
-- Row count in stage
SELECT COUNT(*) AS stage_row_count
FROM tb_reporting.stg_tb_burden_country_clean;
```

Table 1: 1 records

stage_row_count
5120

A reconciliation check confirms that the staging table contains the same number of records as the raw dataset. Both tables report a total of **5,120** rows, indicating that no records were lost during transformation. Performing this validation step reinforces the reliability of the pipeline and demonstrates disciplined data quality verification.

Duplicate Detection

The duplicate detection chunk verifies that the dataset conforms to the intended reporting grain of one record per country per year. By searching for duplicate combinations of country and year in the staging table, the code confirms that the dataset supports a dimensional fact table structure without violating its

grain. Ensuring grain integrity is critical in analytical modeling because duplicates would distort aggregated metrics and produce misleading reporting results.

```sql
-- Duplicate business grain check
SELECT country_name, year, COUNT(*) AS duplicate_count
FROM tb_reporting.stg_tb_burden_country_clean
GROUP BY country_name, year
HAVING COUNT(*) > 1;
```

Table 2: 0 records

country_name	year	duplicate_count

An additional validation step examines whether any duplicate records exist at the intended reporting grain of one row per country per year. The query returns **zero** duplicate records, confirming that the dataset maintains the expected granularity and that no unintended duplication occurred during cleaning.

Missing Value Validation

The missing value validation chunk checks whether any records are missing the key fields that define the dataset's reporting grain, specifically the country identifier or the reporting year. Because these fields are essential for joining dimension tables and maintaining fact table uniqueness, their absence would compromise the integrity of the reporting model. This validation step demonstrates awareness of the importance of key attributes in dimensional modeling and helps guarantee that the dataset is structurally complete before loading it into the analytical schema.

```sql
-- Missing required grain fields
SELECT *
FROM tb_reporting.stg_tb_burden_country_clean
WHERE country_name IS NULL OR year IS NULL;
```

Table 3: 0 records

country_name	year	[illegible]

Data quality checks confirm that all records contain values for the critical grain-defining attributes: country name and reporting year. The absence of missing values in these fields ensures that each record can be accurately linked to its respective dimensions in the reporting model.

Dimension Tables

Country dimension

The country dimension creation chunk builds a lookup table that stores descriptive attributes related to each country in the dataset. Instead of repeating country names, ISO codes, and regional classifications within the fact table, these attributes are centralized in a dedicated dimension. This design reduces redundancy, simplifies reporting queries, and allows analysts to filter and group results by geography. The dimension also supports analytical use cases such as regional trend comparisons and country-level reporting.

```sql
CREATE TABLE tb_reporting.dim_country (
    country_key BIGSERIAL PRIMARY KEY,
    country_name VARCHAR(150) NOT NULL,
    iso2_code VARCHAR(2),
    iso3_code VARCHAR(3),
    iso_numeric_code INT,
    region VARCHAR(100),
    CONSTRAINT uq_dim_country UNIQUE (country_name, iso3_code)
);
```

A dedicated country dimension table is constructed to store geographic attributes such as country name, ISO codes, and world region. Separating descriptive information from measurable metrics reduces redundancy and allows the same country attributes to be reused across multiple analytical queries.

Date dimension

The date dimension creation chunk constructs a time dimension aligned with the dataset's annual reporting structure. Although the dataset records only yearly observations, the dimension enriches the time context by including attributes such as the reporting year and decade grouping. This allows analysts to perform time-based aggregations and trend analysis more easily. Incorporating a date dimension reflects standard data warehouse design practices and prepares the dataset for dashboard visualizations and longitudinal analysis.

```sql
CREATE TABLE tb_reporting.dim_date (
    date_key INT PRIMARY KEY,
    year INT NOT NULL,
    decade INT NOT NULL,
    year_label VARCHAR(10) NOT NULL,
    CONSTRAINT uq_dim_date UNIQUE (year)
);
```

A time dimension table is introduced to support temporal analysis. Each row represents a reporting year and includes derived attributes such as the decade and formatted year labels. Even though the source dataset is annual, adding these derived attributes expands analytical flexibility for trend analysis and aggregated time comparisons.

Methodology dimension

The methodology dimension creation chunk extracts textual descriptors that explain how TB burden metrics were estimated. Rather than repeating these explanatory strings for every record in the fact table, the methodology descriptions are normalized into their own dimension. This approach improves efficiency and makes it possible to analyze results by methodological approach if needed. It also provides transparency for analysts and stakeholders who want to understand the estimation methods behind the reported indicators.

```sql
CREATE TABLE tb_reporting.dim_methodology (
    methodology_key BIGSERIAL PRIMARY KEY,
    prevalence_method VARCHAR(255),
    mortality_method VARCHAR(255),
    incidence_method VARCHAR(255),
    tbhiv_method VARCHAR(255),
    CONSTRAINT uq_dim_methodology UNIQUE (
        prevalence_method,
```

```
        mortality_method,
        incidence_method,
        tbhiv_method
    )
);
```

Methodology descriptions associated with prevalence, incidence, mortality, and TB-HIV estimation methods are normalized into a separate dimension. Storing these textual descriptors in their own table prevents repeated storage within the fact table and improves interpretability of the metrics.

Fact Table

The fact table creation chunk defines the central analytical table that stores all quantitative TB burden measures. This table contains the numeric indicators for prevalence, incidence, mortality, and related measures while linking to the country, date, and methodology dimensions through foreign keys. The design enforces the reporting grain of one row per country per year and provides the foundation for all downstream analytical queries and dashboards. In dimensional modeling, the fact table serves as the primary structure from which aggregated metrics and insights are derived.

```
CREATE TABLE tb_reporting.fact_tb_burden (
    tb_burden_key BIGSERIAL PRIMARY KEY,
    country_key BIGINT NOT NULL,
    date_key INT NOT NULL,
    methodology_key BIGINT,

    estimated_total_population BIGINT,

    prevalence_per_100k NUMERIC(18,2),
    prevalence_per_100k_low NUMERIC(18,2),
    prevalence_per_100k_high NUMERIC(18,2),
    prevalence_total NUMERIC(18,2),
    prevalence_total_low NUMERIC(18,2),
    prevalence_total_high NUMERIC(18,2),

    mortality_excl_hiv_per_100k NUMERIC(18,2),
    mortality_excl_hiv_per_100k_low NUMERIC(18,2),
    mortality_excl_hiv_per_100k_high NUMERIC(18,2),
    deaths_excl_hiv NUMERIC(18,2),
    deaths_excl_hiv_low NUMERIC(18,2),
    deaths_excl_hiv_high NUMERIC(18,2),

    mortality_hiv_pos_per_100k NUMERIC(18,2),
    mortality_hiv_pos_per_100k_low NUMERIC(18,2),
    mortality_hiv_pos_per_100k_high NUMERIC(18,2),
    deaths_hiv_pos NUMERIC(18,2),
    deaths_hiv_pos_low NUMERIC(18,2),
    deaths_hiv_pos_high NUMERIC(18,2),

    incidence_per_100k NUMERIC(18,2),
    incidence_per_100k_low NUMERIC(18,2),
    incidence_per_100k_high NUMERIC(18,2),
    incident_cases NUMERIC(18,2),
```

```
    incident_cases_low NUMERIC(18,2),
    incident_cases_high NUMERIC(18,2),

    hiv_in_incident_tb_percent NUMERIC(18,2),
    hiv_in_incident_tb_percent_low NUMERIC(18,2),
    hiv_in_incident_tb_percent_high NUMERIC(18,2),

    incidence_hiv_pos_per_100k NUMERIC(18,2),
    incidence_hiv_pos_per_100k_low NUMERIC(18,2),
    incidence_hiv_pos_per_100k_high NUMERIC(18,2),
    incidence_hiv_pos NUMERIC(18,2),
    incidence_hiv_pos_low NUMERIC(18,2),
    incidence_hiv_pos_high NUMERIC(18,2),

    case_detection_rate_percent NUMERIC(18,2),
    case_detection_rate_percent_low NUMERIC(18,2),
    case_detection_rate_percent_high NUMERIC(18,2),

    CONSTRAINT fk_fact_tb_country
        FOREIGN KEY (country_key)
        REFERENCES tb_reporting.dim_country(country_key),
    CONSTRAINT fk_fact_tb_date
        FOREIGN KEY (date_key)
        REFERENCES tb_reporting.dim_date(date_key),
    CONSTRAINT fk_fact_tb_methodology
        FOREIGN KEY (methodology_key)
        REFERENCES tb_reporting.dim_methodology(methodology_key),
    CONSTRAINT uq_fact_tb_burden UNIQUE (country_key, date_key)
);
```

The central fact table is created to store all numerical TB burden indicators. Foreign keys reference the country, date, and methodology dimensions, forming the core star schema structure. Constraints enforce uniqueness for each country-year combination, ensuring that the analytical grain of the dataset remains consistent.

Load Dimensions

Country dimension

The country dimension loading chunk populates the country lookup table using distinct country attributes extracted from the staging dataset. By inserting only unique combinations of country names, codes, and regions, the process ensures that each country appears once in the dimension table. This loading step creates a clean reference structure that supports geographic filtering and hierarchical reporting across regions and countries.

```
INSERT INTO tb_reporting.dim_country (
    country_name,
    iso2_code,
    iso3_code,
    iso_numeric_code,
    region
)
```

```sql
SELECT DISTINCT
    country_name,
    iso2_code,
    iso3_code,
    iso_numeric_code,
    region
FROM tb_reporting.stg_tb_burden_country_clean
WHERE country_name IS NOT NULL
ORDER BY country_name;
```

Distinct country records are extracted from the staging table and inserted into the country dimension. The resulting table provides a standardized lookup structure for geographic reporting and supports filtering or grouping of metrics by country or region.

Date dimension

The date dimension loading chunk inserts distinct reporting years into the date dimension while also deriving additional attributes such as decade groupings. Even though the dataset operates at an annual level, enriching the time dimension in this way supports broader trend analysis and simplifies reporting logic in dashboards or analytical queries. This step finalizes the time dimension used throughout the reporting model.

```sql
INSERT INTO tb_reporting.dim_date (
    date_key,
    year,
    decade,
    year_label
)
SELECT DISTINCT
    year AS date_key,
    year,
    (year / 10) * 10 AS decade,
    year::VARCHAR(10) AS year_label
FROM tb_reporting.stg_tb_burden_country_clean
WHERE year IS NOT NULL
ORDER BY year;
```

Distinct reporting years are loaded into the date dimension along with their derived decade attributes. This time reference enables consistent temporal joins and facilitates year-over-year or decade-level reporting.

Methodology dimension

The methodology dimension loading chunk inserts unique combinations of methodological descriptors from the staging dataset into the methodology dimension table. This process removes duplicate text entries and ensures that each distinct methodological approach is stored only once. The resulting dimension allows analytical queries to reference methodology categories through compact foreign keys rather than repeated text values.

```sql
INSERT INTO tb_reporting.dim_methodology (
    prevalence_method,
    mortality_method,
```

```
    incidence_method,
    tbhiv_method
)
SELECT DISTINCT
    prevalence_method,
    mortality_method,
    incidence_method,
    tbhiv_method
FROM tb_reporting.stg_tb_burden_country_clean
ORDER BY 1,2,3,4;
```

Unique combinations of methodology descriptors are inserted into the methodology dimension. By eliminating duplicate descriptions, the table remains compact while still capturing the methodological context associated with each measurement.

Load Fact Table

The fact table loading chunk performs the core dimensional integration step by joining the cleaned staging table with each dimension table to resolve surrogate keys. Once the keys are resolved, the code inserts all numeric TB burden measures into the fact table along with references to their corresponding country, date, and methodology records. This process transforms the cleaned staging data into a fully structured star schema that supports efficient analytical reporting.

```
INSERT INTO tb_reporting.fact_tb_burden (
    country_key,
    date_key,
    methodology_key,
    estimated_total_population,
    prevalence_per_100k,
    prevalence_per_100k_low,
    prevalence_per_100k_high,
    prevalence_total,
    prevalence_total_low,
    prevalence_total_high,
    mortality_excl_hiv_per_100k,
    mortality_excl_hiv_per_100k_low,
    mortality_excl_hiv_per_100k_high,
    deaths_excl_hiv,
    deaths_excl_hiv_low,
    deaths_excl_hiv_high,
    mortality_hiv_pos_per_100k,
    mortality_hiv_pos_per_100k_low,
    mortality_hiv_pos_per_100k_high,
    deaths_hiv_pos,
    deaths_hiv_pos_low,
    deaths_hiv_pos_high,
    incidence_per_100k,
    incidence_per_100k_low,
    incidence_per_100k_high,
    incident_cases,
    incident_cases_low,
    incident_cases_high,
```

```
    hiv_in_incident_tb_percent,
    hiv_in_incident_tb_percent_low,
    hiv_in_incident_tb_percent_high,
    incidence_hiv_pos_per_100k,
    incidence_hiv_pos_per_100k_low,
    incidence_hiv_pos_per_100k_high,
    incidence_hiv_pos,
    incidence_hiv_pos_low,
    incidence_hiv_pos_high,
    case_detection_rate_percent,
    case_detection_rate_percent_low,
    case_detection_rate_percent_high
)
SELECT
    c.country_key,
    d.date_key,
    m.methodology_key,
    s.estimated_total_population,
    s.prevalence_per_100k,
    s.prevalence_per_100k_low,
    s.prevalence_per_100k_high,
    s.prevalence_total,
    s.prevalence_total_low,
    s.prevalence_total_high,
    s.mortality_excl_hiv_per_100k,
    s.mortality_excl_hiv_per_100k_low,
    s.mortality_excl_hiv_per_100k_high,
    s.deaths_excl_hiv,
    s.deaths_excl_hiv_low,
    s.deaths_excl_hiv_high,
    s.mortality_hiv_pos_per_100k,
    s.mortality_hiv_pos_per_100k_low,
    s.mortality_hiv_pos_per_100k_high,
    s.deaths_hiv_pos,
    s.deaths_hiv_pos_low,
    s.deaths_hiv_pos_high,
    s.incidence_per_100k,
    s.incidence_per_100k_low,
    s.incidence_per_100k_high,
    s.incident_cases,
    s.incident_cases_low,
    s.incident_cases_high,
    s.hiv_in_incident_tb_percent,
    s.hiv_in_incident_tb_percent_low,
    s.hiv_in_incident_tb_percent_high,
    s.incidence_hiv_pos_per_100k,
    s.incidence_hiv_pos_per_100k_low,
    s.incidence_hiv_pos_per_100k_high,
    s.incidence_hiv_pos,
    s.incidence_hiv_pos_low,
    s.incidence_hiv_pos_high,
    s.case_detection_rate_percent,
    s.case_detection_rate_percent_low,
```

```sql
    s.case_detection_rate_percent_high
FROM tb_reporting.stg_tb_burden_country_clean s
JOIN tb_reporting.dim_country c
    ON s.country_name = c.country_name
   AND COALESCE(s.iso3_code, '') = COALESCE(c.iso3_code, '')
JOIN tb_reporting.dim_date d
    ON s.year = d.year
JOIN tb_reporting.dim_methodology m
    ON COALESCE(s.prevalence_method, '') = COALESCE(m.prevalence_method, '')
   AND COALESCE(s.mortality_method, '') = COALESCE(m.mortality_method, '')
   AND COALESCE(s.incidence_method, '') = COALESCE(m.incidence_method, '')
   AND COALESCE(s.tbhiv_method, '') = COALESCE(m.tbhiv_method, '');
```

The cleaned staging dataset is joined to the dimension tables in order to populate the fact table. Surrogate keys are resolved for each dimension, and all numeric TB burden metrics are inserted into the central fact structure. At this stage, the project transitions from a cleaned dataset into a fully operational dimensional warehouse model.

Performance Indexes

The indexing chunk adds database indexes to key columns used in joins and filters across the fact and dimension tables. Indexes improve query performance by allowing the database engine to locate records more efficiently during reporting operations. Creating indexes on commonly queried attributes such as country, region, and year ensures that analytical queries and dashboards can retrieve results quickly even as the dataset grows in size.

```sql
CREATE INDEX idx_dim_country_name ON tb_reporting.dim_country(country_name);
```

```sql
CREATE INDEX idx_dim_country_region ON tb_reporting.dim_country(region);
```

```sql
CREATE INDEX idx_dim_country_iso3 ON tb_reporting.dim_country(iso3_code);
```

```sql
CREATE INDEX idx_fact_tb_country_key ON tb_reporting.fact_tb_burden(country_key);
```

```sql
CREATE INDEX idx_fact_tb_date_key ON tb_reporting.fact_tb_burden(date_key);
```

```sql
CREATE INDEX idx_fact_tb_methodology_key ON tb_reporting.fact_tb_burden(methodology_key);
```

```sql
CREATE INDEX idx_fact_tb_country_date ON tb_reporting.fact_tb_burden(country_key, date_key);
```

Indexes are created on frequently queried columns such as country identifiers, regional attributes, and foreign keys within the fact table. These indexes improve query execution speed and ensure that the model performs efficiently when accessed by reporting tools or analytical queries.

Reporting Views

Country-Year Reporting View

The detailed reporting view creation chunk builds a user-friendly view that joins the fact table with its related dimension tables. This view presents a flattened dataset containing both descriptive attributes and

analytical measures, making it easier for analysts or business intelligence tools to query the data without manually writing complex joins. By exposing a clean reporting layer, the project improves accessibility for downstream users.

```sql
CREATE VIEW tb_reporting.vw_tb_country_year_report AS
SELECT
    f.tb_burden_key,
    d.year,
    d.decade,
    c.country_name,
    c.iso2_code,
    c.iso3_code,
    c.iso_numeric_code,
    c.region,
    m.prevalence_method,
    m.mortality_method,
    m.incidence_method,
    m.tbhiv_method,
    f.estimated_total_population,
    f.prevalence_per_100k,
    f.prevalence_per_100k_low,
    f.prevalence_per_100k_high,
    f.prevalence_total,
    f.prevalence_total_low,
    f.prevalence_total_high,
    f.mortality_excl_hiv_per_100k,
    f.mortality_excl_hiv_per_100k_low,
    f.mortality_excl_hiv_per_100k_high,
    f.deaths_excl_hiv,
    f.deaths_excl_hiv_low,
    f.deaths_excl_hiv_high,
    f.mortality_hiv_pos_per_100k,
    f.mortality_hiv_pos_per_100k_low,
    f.mortality_hiv_pos_per_100k_high,
    f.deaths_hiv_pos,
    f.deaths_hiv_pos_low,
    f.deaths_hiv_pos_high,
    f.incidence_per_100k,
    f.incidence_per_100k_low,
    f.incidence_per_100k_high,
    f.incident_cases,
    f.incident_cases_low,
    f.incident_cases_high,
    f.hiv_in_incident_tb_percent,
    f.hiv_in_incident_tb_percent_low,
    f.hiv_in_incident_tb_percent_high,
    f.incidence_hiv_pos_per_100k,
    f.incidence_hiv_pos_per_100k_low,
    f.incidence_hiv_pos_per_100k_high,
    f.incidence_hiv_pos,
    f.incidence_hiv_pos_low,
    f.incidence_hiv_pos_high,
    f.case_detection_rate_percent,
    f.case_detection_rate_percent_low,
```

```sql
    f.case_detection_rate_percent_high
FROM tb_reporting.fact_tb_burden f
JOIN tb_reporting.dim_country c
    ON f.country_key = c.country_key
JOIN tb_reporting.dim_date d
    ON f.date_key = d.date_key
LEFT JOIN tb_reporting.dim_methodology m
    ON f.methodology_key = m.methodology_key;
```

A reporting view is created that joins the fact table with all associated dimensions to produce a flattened dataset suitable for analysis and visualization. The view simplifies access for analysts and business intelligence platforms by removing the need to repeatedly construct complex joins.

Regional Summary View

The regional summary view creation chunk produces an aggregated reporting structure that summarizes TB burden metrics by region and year. Instead of requiring users to calculate totals and averages manually, the view provides precomputed metrics that can be used directly in dashboards or reports. This design supports high-level monitoring of regional trends and simplifies performance comparisons across geographic areas.

```sql
CREATE VIEW tb_reporting.vw_tb_region_year_summary AS
SELECT
    d.year,
    c.region,
    COUNT(*) AS country_rows,
    SUM(f.estimated_total_population) AS total_population,
    AVG(f.prevalence_per_100k) AS avg_prevalence_per_100k,
    AVG(f.mortality_excl_hiv_per_100k) AS avg_mortality_excl_hiv_per_100k,
    AVG(f.incidence_per_100k) AS avg_incidence_per_100k,
    AVG(f.case_detection_rate_percent) AS avg_case_detection_rate_percent,
    SUM(f.incident_cases) AS total_incident_cases,
    SUM(f.deaths_excl_hiv) AS total_deaths_excl_hiv,
    SUM(f.deaths_hiv_pos) AS total_deaths_hiv_pos
FROM tb_reporting.fact_tb_burden f
JOIN tb_reporting.dim_country c
    ON f.country_key = c.country_key
JOIN tb_reporting.dim_date d
    ON f.date_key = d.date_key
GROUP BY d.year, c.region;
```

An aggregated reporting view summarizes TB burden metrics at the region-year level. This structure provides precomputed totals and averages, enabling quick access to high-level indicators frequently used in dashboards and strategic reporting.

Validation Queries

Row Count Reconciliation

The validation query chunk performs final reconciliation checks across the raw, staging, and fact tables. By confirming that row counts match and that dimension tables contain the expected number of records, the pipeline verifies that the entire transformation process completed successfully. This final validation step provides confidence that the reporting model accurately reflects the original dataset.

```sql
SELECT COUNT(*) AS raw_count
FROM tb_reporting.raw_tb_burden_country;
```

Table 4: 1 records

raw_count
5120

```sql
SELECT COUNT(*) AS stage_count
FROM tb_reporting.stg_tb_burden_country_clean;
```

Table 5: 1 records

stage_count
5120

```sql
SELECT COUNT(*) AS fact_count
FROM tb_reporting.fact_tb_burden;
```

Table 6: 1 records

fact_count
5120

```sql
SELECT COUNT(*) AS country_count
FROM tb_reporting.dim_country;
```

Table 7: 1 records

country_count
219

```sql
SELECT COUNT(*) AS year_count
FROM tb_reporting.dim_date;
```

Table 8: 1 records

year_count
24

End-to-end validation confirms that the entire pipeline loaded successfully. The database reports **5,120** rows in the raw dataset, **5,120** rows in the staging layer, and **5,120** rows in the fact table, indicating full reconciliation across all stages. Dimension tables contain **219** countries and **24** reporting years, which aligns with the expected structure of the global dataset.

Fact Table Integrity Check

The fact table integrity check searches for duplicate records at the fact table grain after the dimensional loading process. Because the fact table is designed to contain exactly one record per country per year, the presence of duplicates would indicate a problem in the loading logic. This check confirms that the fact table maintains its intended grain and that analytical results derived from it will remain accurate.

```
SELECT country_key, date_key, COUNT(*) AS duplicate_count
FROM tb_reporting.fact_tb_burden
GROUP BY country_key, date_key
HAVING COUNT(*) > 1;
```

Table 9: 0 records

country_key	date_key	duplicate_count

A final integrity check confirms that the fact table contains **no duplicate** combinations of country and year. Maintaining this strict grain ensures that aggregated metrics and trend analyses remain accurate.

Sample Reporting Queries

Country Trend Analysis

The country trend analysis query demonstrates how the reporting model can be used to analyze TB burden trends for a specific country over time. By retrieving indicators such as incidence, case detection rate, and deaths for each year, the query illustrates the analytical value of the star schema. This example highlights how the model supports time-series analysis and country-level monitoring.

```
SELECT
    c.country_name,
    d.year,
    f.incidence_per_100k,
    f.case_detection_rate_percent,
    f.deaths_excl_hiv
FROM tb_reporting.fact_tb_burden f
JOIN tb_reporting.dim_country c ON f.country_key = c.country_key
JOIN tb_reporting.dim_date d ON f.date_key = d.date_key
WHERE c.country_name = 'India'
ORDER BY d.year;
```

Table 10: Displaying records 1 - 10

country_name	year	incidence_per_100k	case_detection_rate_percent	deaths_excl_hiv
India	1990	217	81	330000
India	1991	216	81	340000
India	1992	216	57	350000
India	1993	216	54	350000
India	1994	216	55	360000
India	1995	216	59	370000

country_name	year	incidence_per_100k	case_detection_rate_percent	deaths_excl_hiv
India	1996	217	61	370000
India	1997	217	53	380000
India	1998	217	50	390000
India	1999	217	55	400000

Example analytical queries demonstrate the usability of the model by retrieving annual TB metrics for India. The results illustrate year-over-year values for incidence rates, case detection percentages, and mortality counts, confirming that the database supports longitudinal country-level analysis.

Top Countries Analysis

The top countries query ranks countries by the number of incident TB cases in a selected year. This type of analysis helps identify the regions and countries with the highest disease burden and is useful for comparative reporting and prioritization. Demonstrating this query shows how the data model can support ranking and benchmarking analyses.

```sql
SELECT
    d.year,
    c.country_name,
    c.region,
    f.incident_cases,
    f.deaths_excl_hiv
FROM tb_reporting.fact_tb_burden f
JOIN tb_reporting.dim_country c ON f.country_key = c.country_key
JOIN tb_reporting.dim_date d ON f.date_key = d.date_key
WHERE d.year = 2013
ORDER BY f.incident_cases DESC NULLS LAST
LIMIT 10;
```

Table 11: Displaying records 1 - 10

year	country_name	region	incident_cases	deaths_excl_hiv
2013	India	SEA	2100000	240000
2013	China	WPR	980000	41000
2013	Nigeria	AFR	590000	160000
2013	Pakistan	EMR	500000	49000
2013	Indonesia	SEA	460000	64000
2013	South Africa	AFR	450000	25000
2013	Bangladesh	SEA	350000	80000
2013	Philippines	WPR	290000	27000
2013	Democratic Republic of the Congo	AFR	220000	46000
2013	Ethiopia	AFR	210000	30000

Ranking queries identify the countries with the highest number of TB incident cases in a given year. Results show India leading with approximately **2.1 million cases**, followed by China and Nigeria. Such comparisons highlight the ability of the reporting model to support prioritization and benchmarking.

Regional Summary Analysis

The regional totals query retrieves aggregate TB burden metrics across regions and years. By summarizing population, incident cases, and deaths, this query supports high-level reporting used by policymakers and public health analysts. The results are suitable for dashboards that monitor regional trends over time.

```sql
SELECT
  year,
  region,
  total_population,
  total_incident_cases,
  total_deaths_excl_hiv,
  total_deaths_hiv_pos
FROM tb_reporting.vw_tb_region_year_summary
ORDER BY year DESC, region;
```

Table 12: Displaying records 1 - 10

year	region	total_population	total_incident_cases	total_deaths_excl_hiv	total_deaths_hiv_pos
2013	AFR	927370706	2595728.0	388509.40	295534.00
2013	AMR	970820511	285212.8	16963.06	6142.00
2013	EMR	616906419	741950.0	90290.20	2933.56
2013	EUR	907052722	359760.1	37245.12	3785.00
2013	SEA	1855067723	3364040.0	431785.60	48604.00
2013	WPR	1858410405	1617034.4	108138.10	4777.60
2012	AFR	903366645	2599913.0	385896.00	305475.00
2012	AMR	961102884	286149.2	17811.31	6597.00
2012	EMR	605753539	732160.0	90388.20	2832.55
2012	EUR	904540315	372612.3	40388.29	4291.00

Aggregated queries summarize total population, incident cases, and deaths by world region. The results indicate particularly high case counts within the Southeast Asia and Africa regions, demonstrating how the model enables large-scale epidemiological comparisons.

Regional Averages Analysis

The regional averages query calculates average prevalence, mortality, and incidence values by region and year. This analysis allows comparisons across geographic regions and supports evaluation of how disease burden evolves over time. Providing this query demonstrates the flexibility of the reporting model in supporting multiple types of analytical perspectives.

```sql
SELECT
  year,
  region,
  avg_prevalence_per_100k,
  avg_mortality_excl_hiv_per_100k,
  avg_incidence_per_100k
FROM tb_reporting.vw_tb_region_year_summary
ORDER BY year, region;
```

Table 13: Displaying records 1 - 10

year	region	avg_prevalence_per_100k	avg_mortality_excl_hiv_per_100k	avg_incidence_per_100k
1990	AFR	417.13043	59.995652	249.63043
1990	AMR	84.42889	5.784444	51.77111
1990	EMR	183.54545	22.324091	103.68182
1990	EUR	84.18113	4.256793	50.21698
1990	SEA	550.30000	83.660000	271.90000
1990	WPR	307.64722	28.359722	156.40556
1991	AFR	407.10870	58.445652	253.82609
1991	AMR	83.11111	5.660667	51.79778
1991	EMR	179.36364	21.298636	103.72727
1991	EUR	91.73774	4.251887	53.68302

Additional queries compute regional averages for prevalence, incidence, and mortality per 100,000 population. These normalized metrics provide a balanced basis for comparing TB burden across geographic regions over time.

Conclusion

This project demonstrates the complete transformation of a complex global health dataset into a structured reporting database designed for analytical exploration and business intelligence reporting. Beginning with a raw CSV dataset containing country-level TB burden metrics, the workflow systematically builds a production-style data pipeline that ingests, validates, cleans, and organizes the data into a PostgreSQL star schema. The resulting model preserves the analytical grain of one record per country per year, ensuring that reporting results remain accurate and consistent across aggregations and trend analyses.

The dimensional model separates descriptive attributes from quantitative measures through dedicated country, date, and methodology dimensions, while the fact table stores all numeric disease indicators required for analytical reporting. This structure reduces redundancy, simplifies query design, and enables efficient aggregation of epidemiological metrics across geographic regions and time periods. By implementing indexes on frequently queried attributes and constructing reporting views that flatten dimensional joins, the database also improves query performance and accessibility for downstream analytical tools.

Data integrity and reliability were emphasized throughout the pipeline. Validation queries confirmed that row counts remained consistent across raw, staging, and fact layers, demonstrating that transformations preserved the full dataset. Additional integrity checks verified that the dimensional grain remained intact and that no duplicate country-year records were introduced during loading. These validation steps reflect common practices used in enterprise data pipelines where maintaining data accuracy and traceability is critical.

The final reporting layer supports a variety of analytical queries, including country-level trend analysis, ranking countries by disease burden, and regional aggregation of incidence and mortality indicators. These capabilities illustrate how the dimensional model enables meaningful comparisons across countries and regions while supporting longitudinal analysis over multiple decades. Such insights can help policymakers, analysts, and researchers better understand patterns in global TB burden and identify areas where public health interventions may be most needed.

Beyond its epidemiological context, the project also serves as a practical demonstration of modern reporting data analytics skills. It highlights the ability to design dimensional data models, implement reproducible analytical pipelines, integrate scripting environments with relational databases, and prepare structured datasets for reporting and visualization. These competencies are essential in many analytics and reporting roles where transforming raw data into reliable, decision-ready information is a core responsibility.